

# Interrupções



# Interrupções

- Até agora, o fluxo do programa é determinado apenas pelo programa principal

Problema.

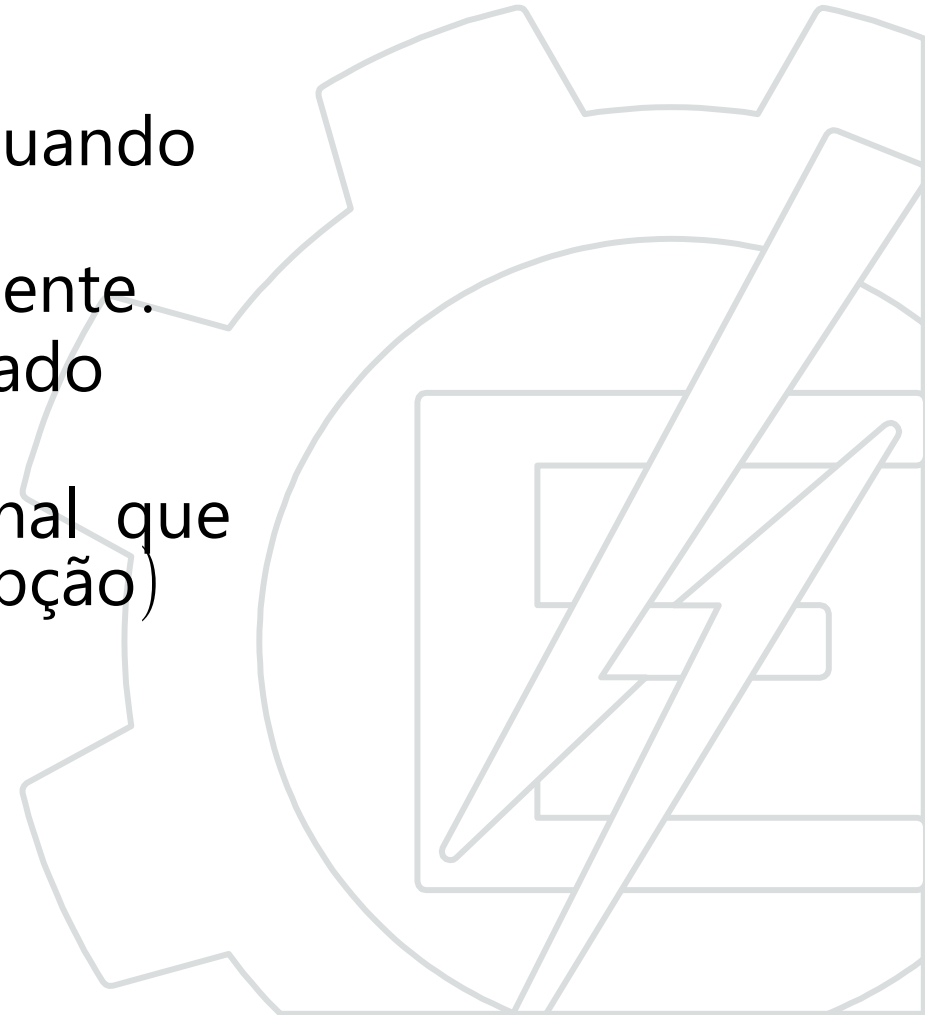
- Alguns periféricos necessitam de um tempo para que possam executar suas tarefas.
- No programa principal este tempo é gasto fazendo nada (loop vazio) para dar um tempo ou esperar uma variável ou flag mudar de estado.
- Esta técnica é denominada **pooling**.

Exemplo:

```
...  
while(PORTA!=0x01); //0 programa espera uma condição  
...
```

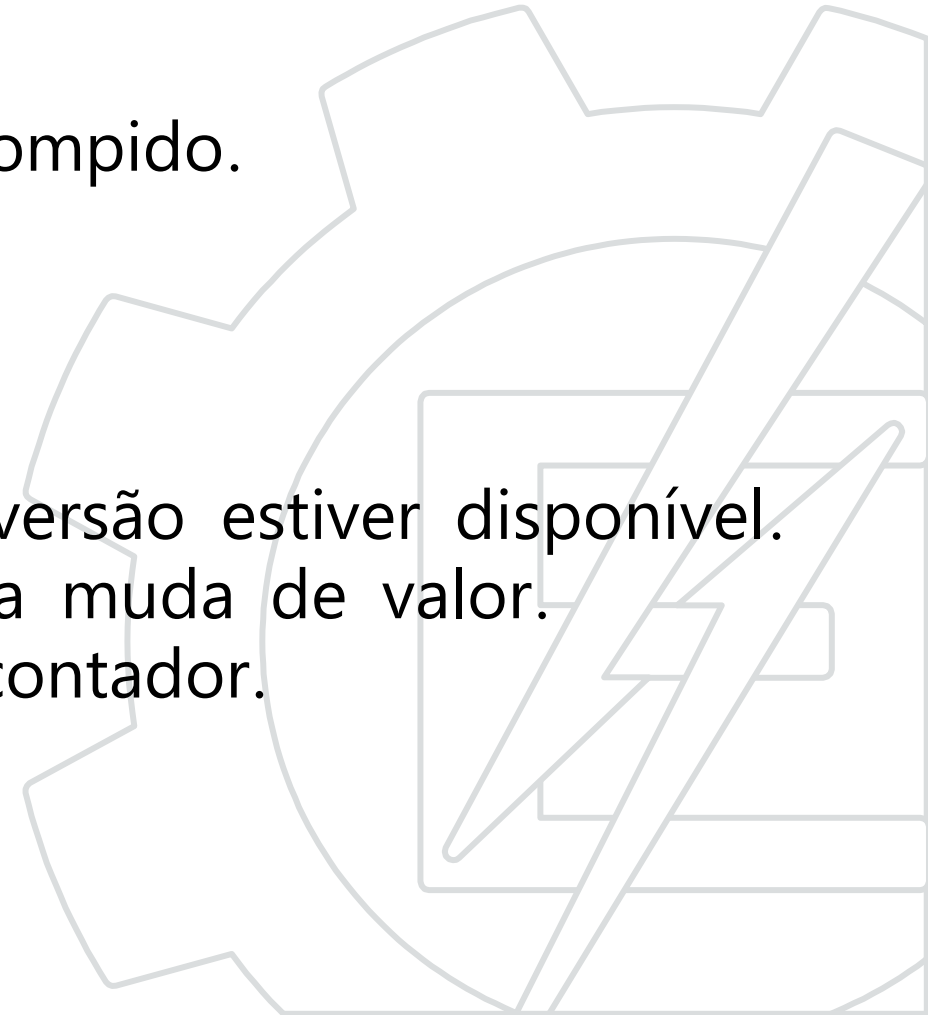
# Interrupções

- Como evitar o pooling:
  - Desenvolver um sistema que nos avise quando uma determinada tarefa acabou.
  - Este sistema deve funcionar automaticamente.
  - O fluxo do programa não deve ser alterado durante a execução da tarefa.
  - O dispositivo/periférico deve gerar um sinal que interrompe o fluxo do programa (interrupção) quando a tarefa terminar.

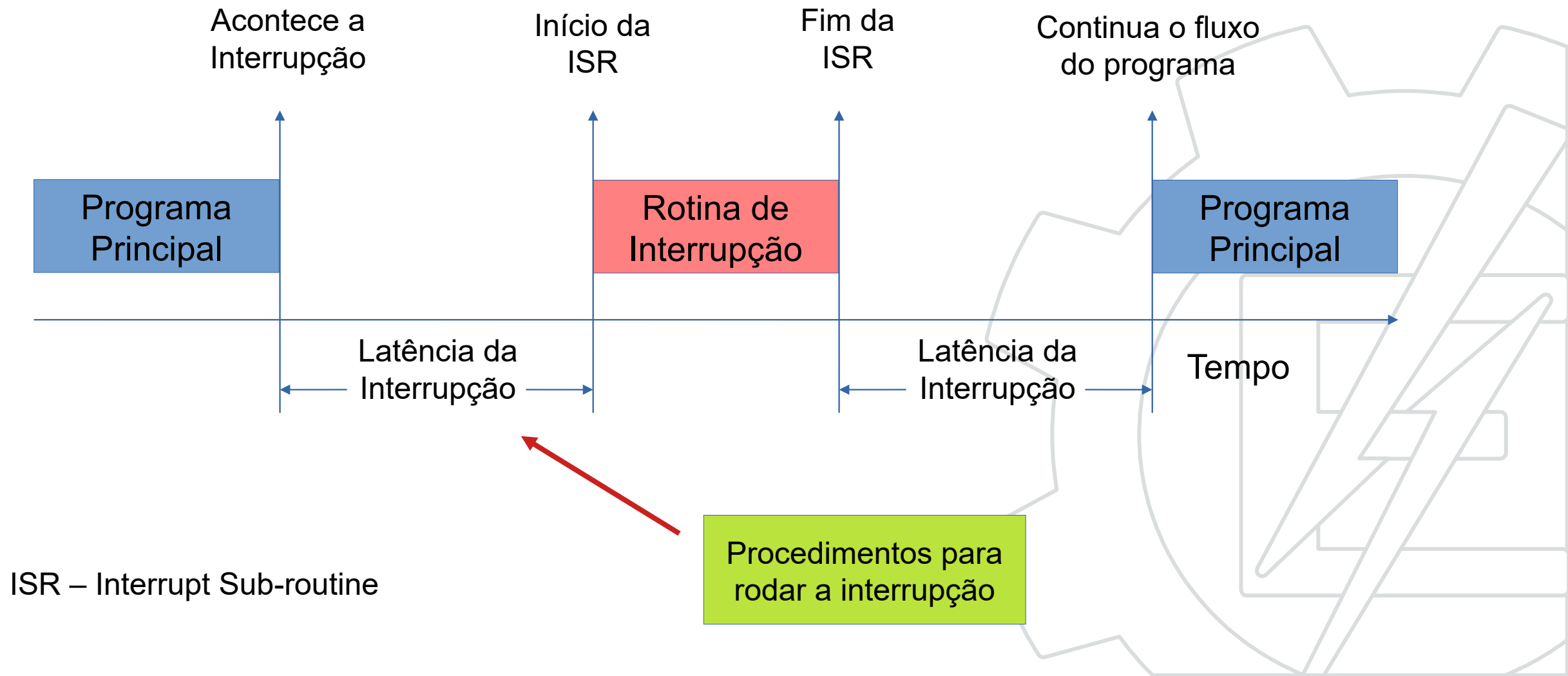


# Interrupções

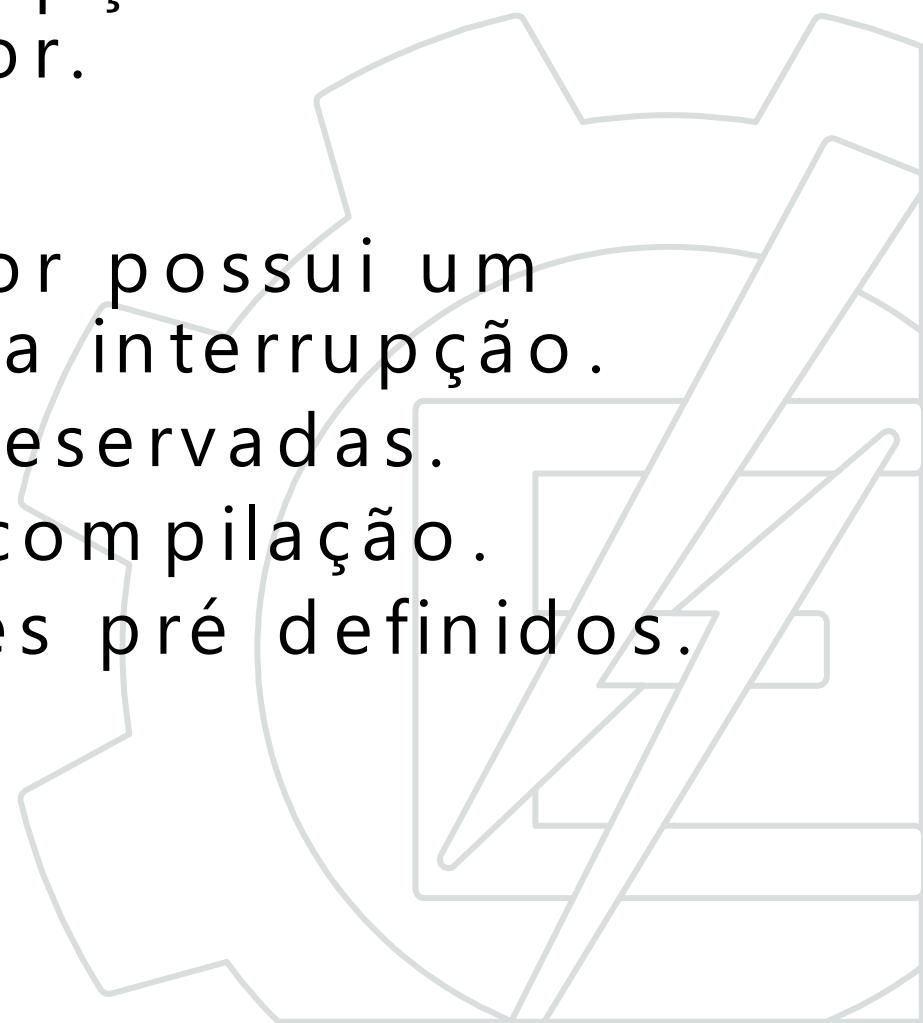
- Quando gerada uma interrupção o programa é paralisado e uma função de interrupção é executada.
- O programa principal não sabe que foi interrompido.
- Exemplos:
  - Conversor AD: quando o resultado da conversão estiver disponível.
  - Porta B: quando algum dos bits na entrada muda de valor.
  - Timer 0: quando ocorre overflow em seu contador.



# Interrupções

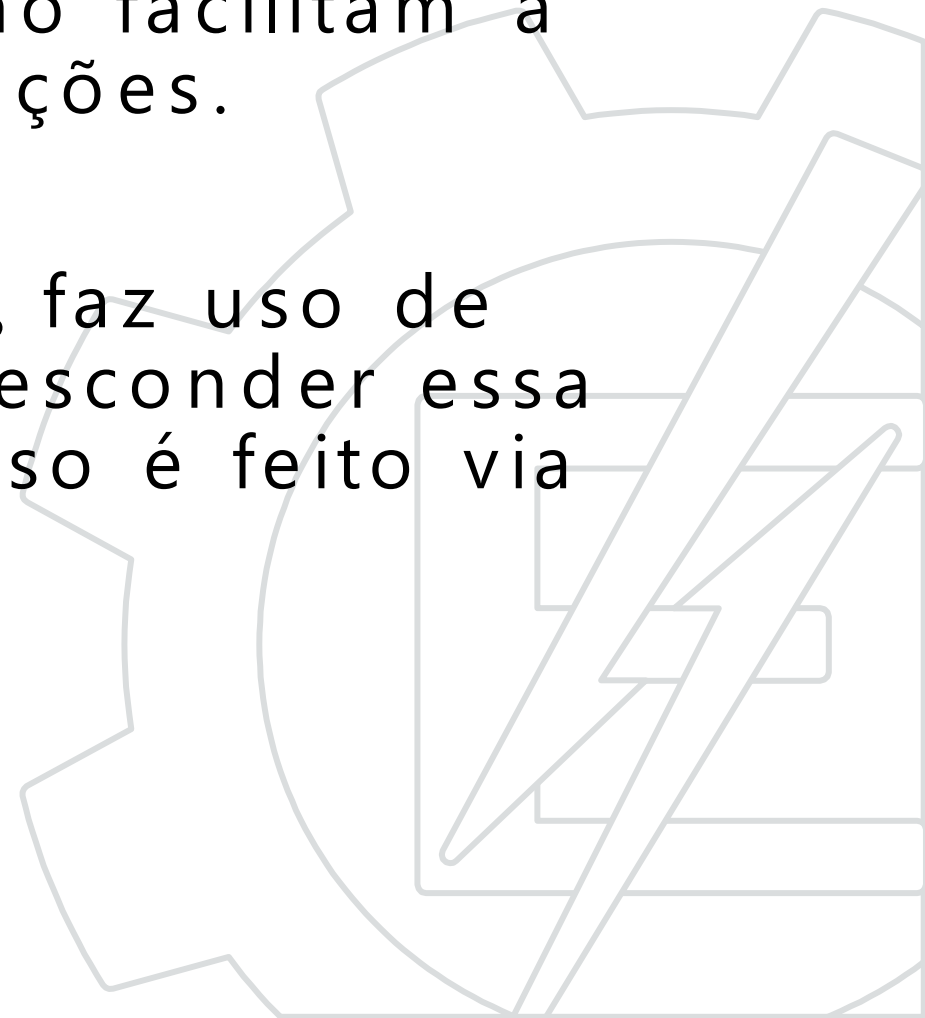


# Interrupções

- A função que irá tratar da interrupção não retorna nem recebe nenhum valor.
  - Cada compilador/microcontrolador possui um modo diferente de implementar a interrupção.
    - Utilização de palavras chaves reservadas.
    - Utilização de diretivas de pré compilação.
    - Utilização de nomes de funções pré definidos.
- 

# Interrupções

- Alguns ambientes de programação facilitam a programação e o uso de interrupções.
- O framework Wiring, do Arduino, faz uso de estruturas de programação para esconder essa complexidade do programador. Isso é feito via função `attachInterrupt()`.



# SDCC (fork do GCC)

- Uso da palavra chave `__interrupt`:

Quando o circuito AD  
ativar a interrupção

```
// Do manual do compilador SDCC temos:  
//      4 -> trata as interrupções do AD  
//      7 -> trata as interrupções da serial  
void NomeDaFuncao(void) __interrupt 4 {  
    //código para tratar a interrupção AD  
}  
void NomeDaFuncao(void) __interrupt 7 {  
    //código para tratar a interrupção da serial  
}
```

Essa função será executada

# XC8

- Uso da palavra chave interrupt:

interrupt antes do nome  
da função

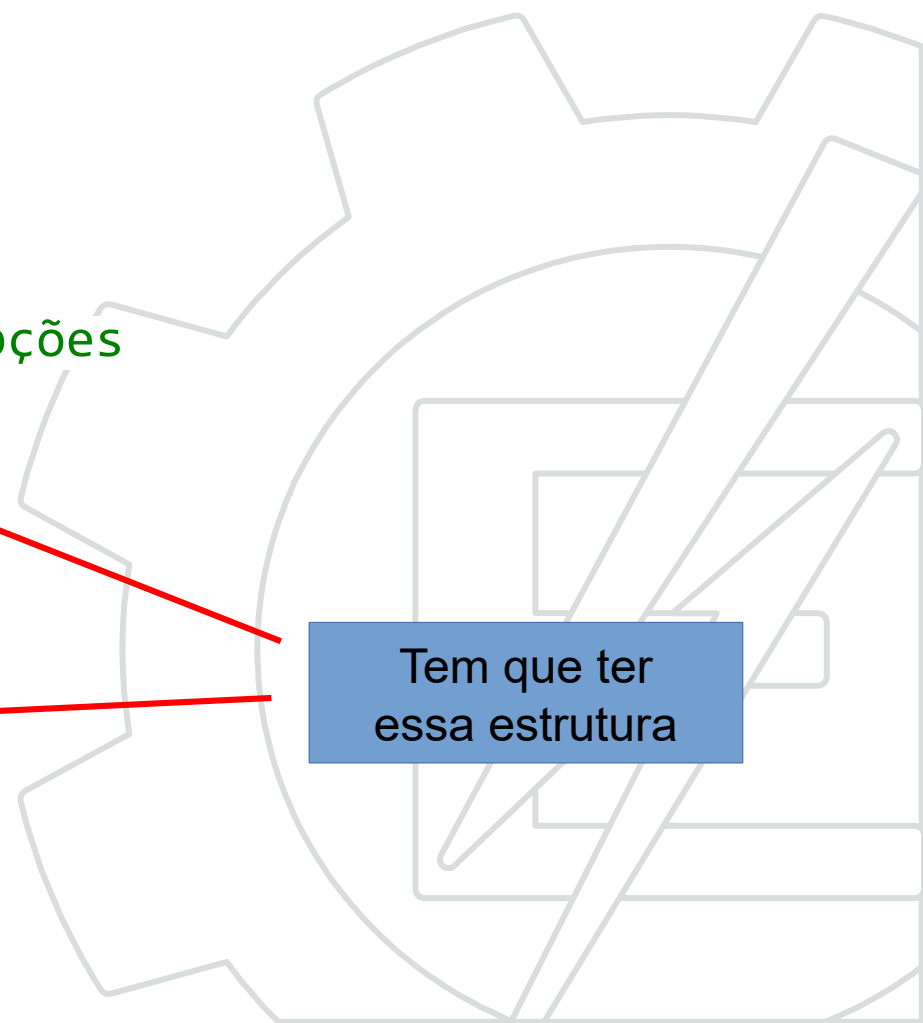
Verifica qual interrupt

```
void interrupt tc_int (void) // função da interrupção
{
    if(INTCONbits.T0IF && INTCONbits.T0IE) {
        // se a flag estiver ativa e a interrupção ligada
        TMR0 -= 250; // reinicia o timer
        INTCONbits.T0IF = 0; // limpa a flag
        PORTB ^= 0x40; // pisca um led
    }
}
```

# C18

- Uso de diretivas de pré compilação

```
//Usando compilador C18
void NomeDaFuncao(void) {
    //código ...
}
//Indicar a posição no vetor de interrupções
#pragma code high_vector=0x08
void interrupt at_high_vector(void){
    _asm GOTO NomeDaFuncao _endasm
}
#pragma code
#pragma interrupt NomeDaFuncao
```



Tem que ter  
essa estrutura

Two red arrows point from the text box to the code: one points to the `GOTO NomeDaFuncao` line, and the other points to the `#pragma interrupt NomeDaFuncao` line.

# Freedom

- Uso de funções pré definidas

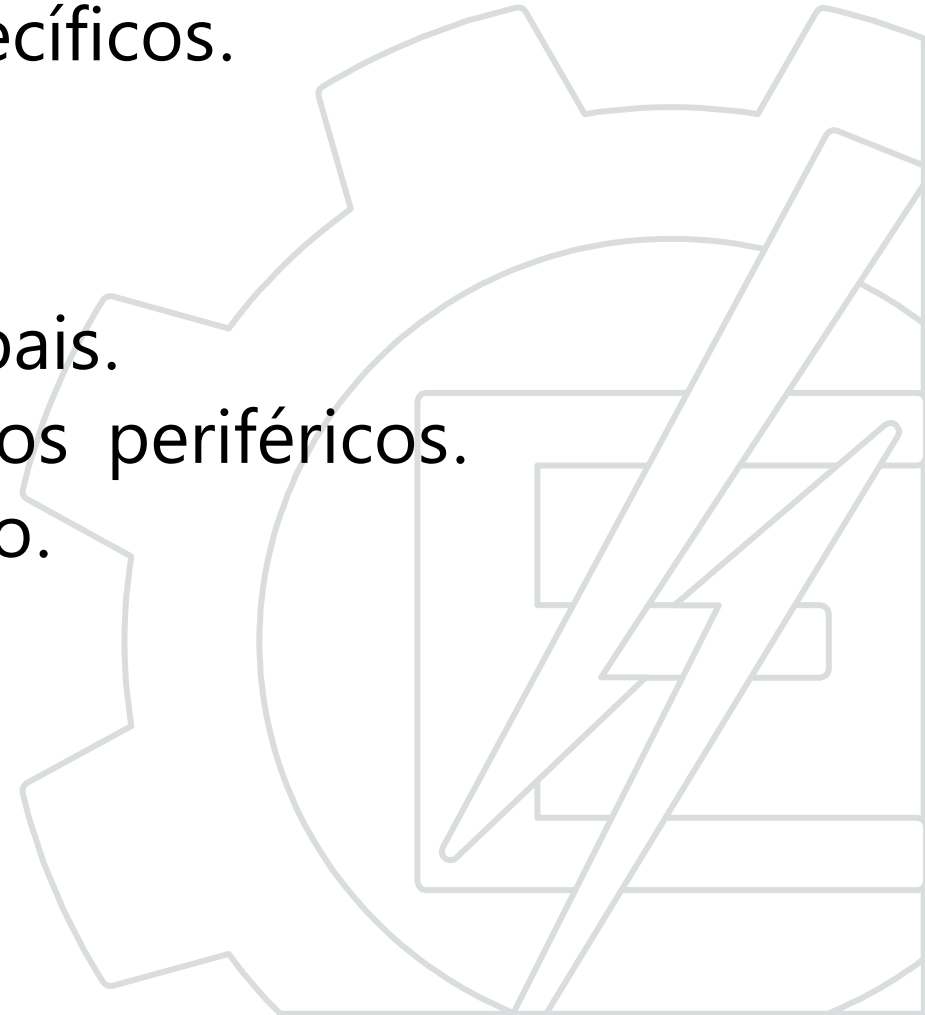
```
// a interrupção da UART 0 sempre é atendida  
// pela função UART0_IRQHandler()  
void UART0_IRQHandler(void) {  
    // só faz um loopback do que recebeu  
    serialSend(serialRead());  
}
```

Essa função é  
do compilador

Ela chama a sua  
função

# Utilizando interrupções

- A interrupção deve estar habilitada para funcionar, isso é feito através de configurações em registros específicos.
- A configuração deve ser feita:
  - Nos registros ativando interrupções globais.
  - Nos registros habilitando interrupções dos periféricos.
  - No periférico que irá gerar a interrupção.

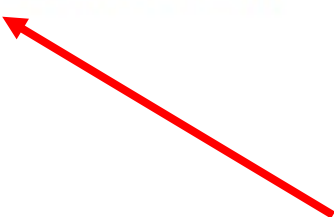


# Exemplo PIC16F628A

## 14.5 Interrupts

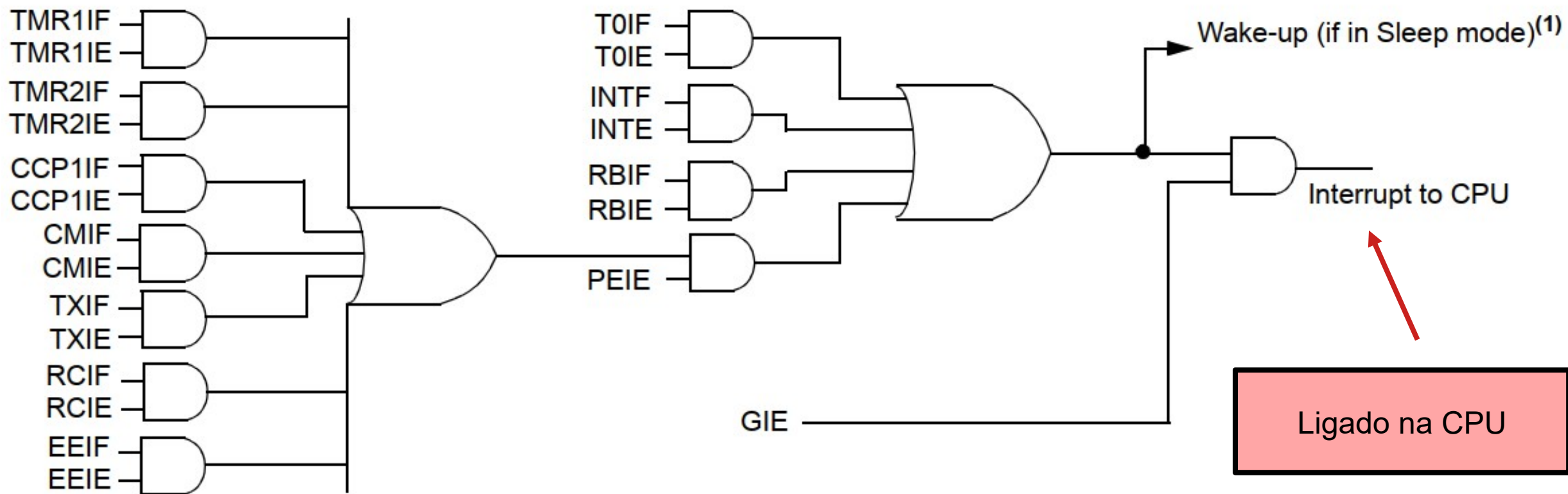
The PIC16F627A/628A/648A has 10 sources of interrupt:

- External Interrupt RB0/INT
- TMR0 Overflow Interrupt
- PORTB Change Interrupts (pins RB<7:4>)
- Comparator Interrupt
- USART Interrupt TX
- USART Interrupt RX
- CCP Interrupt
- TMR1 Overflow Interrupt
- TMR2 Match Interrupt
- Data EEPROM Interrupt

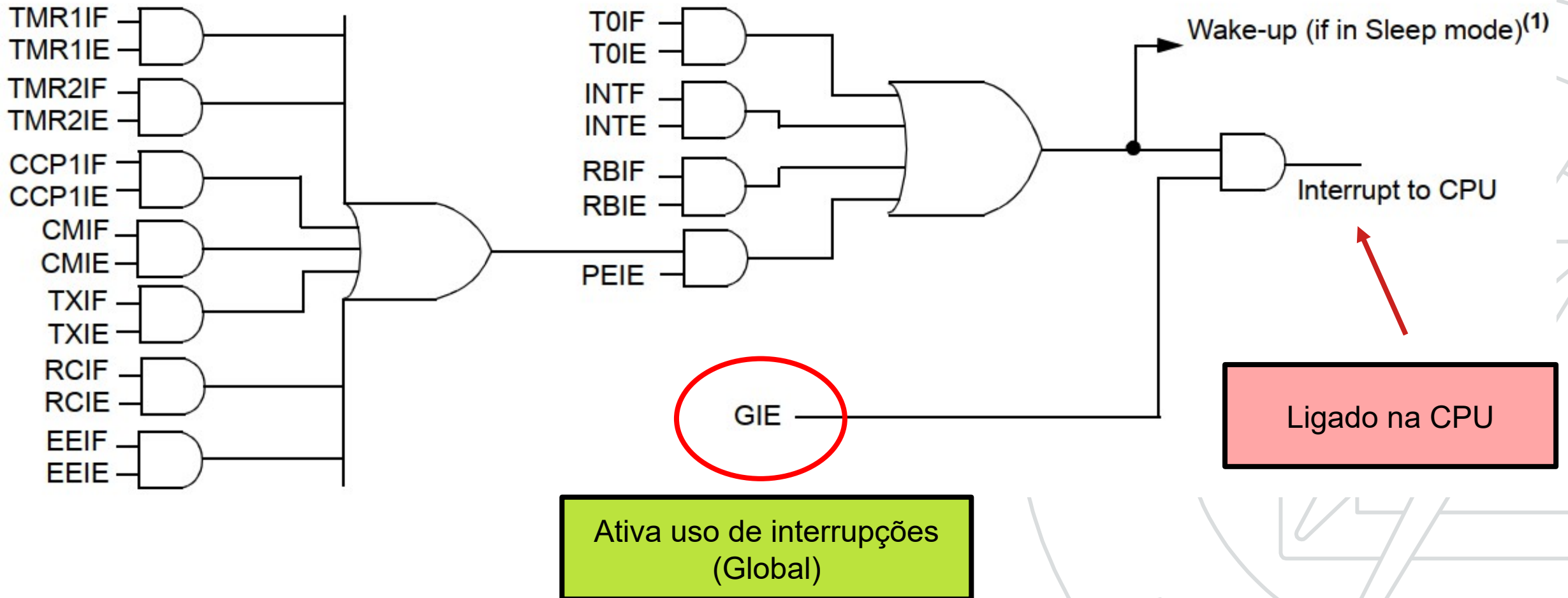


O PIC16F628A possui 10 fontes de interrupção

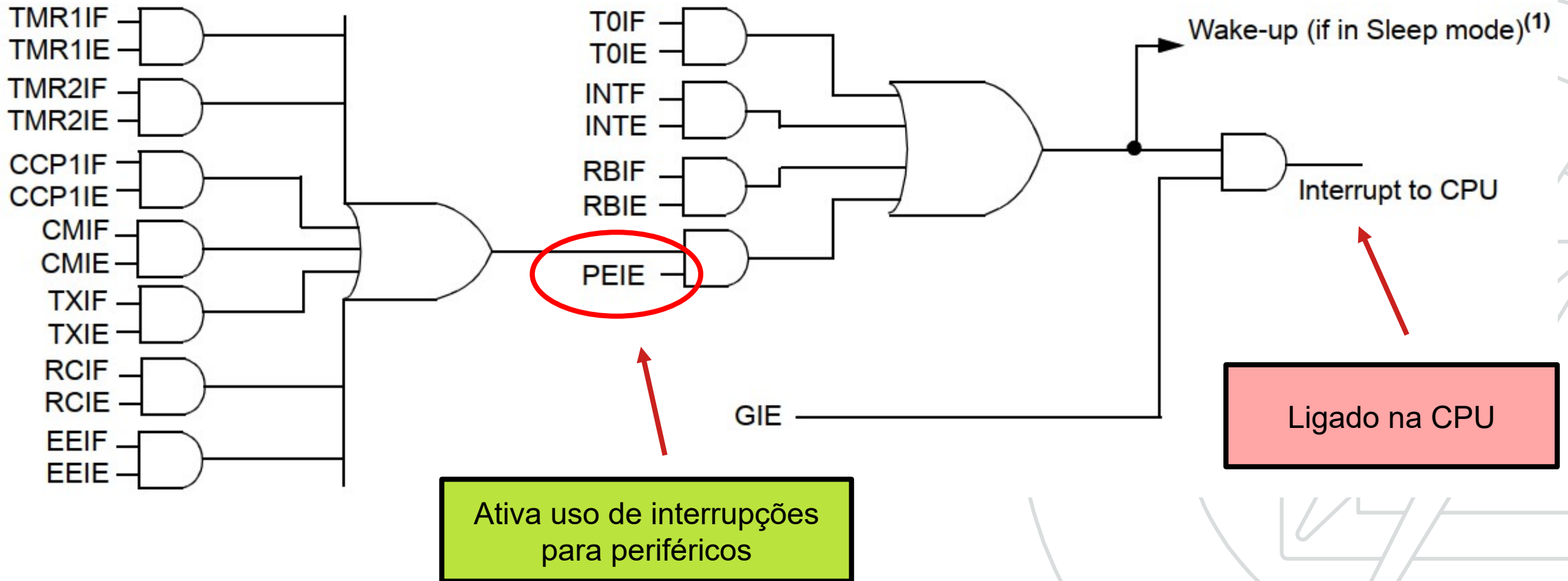
# Exemplo PIC16F628A



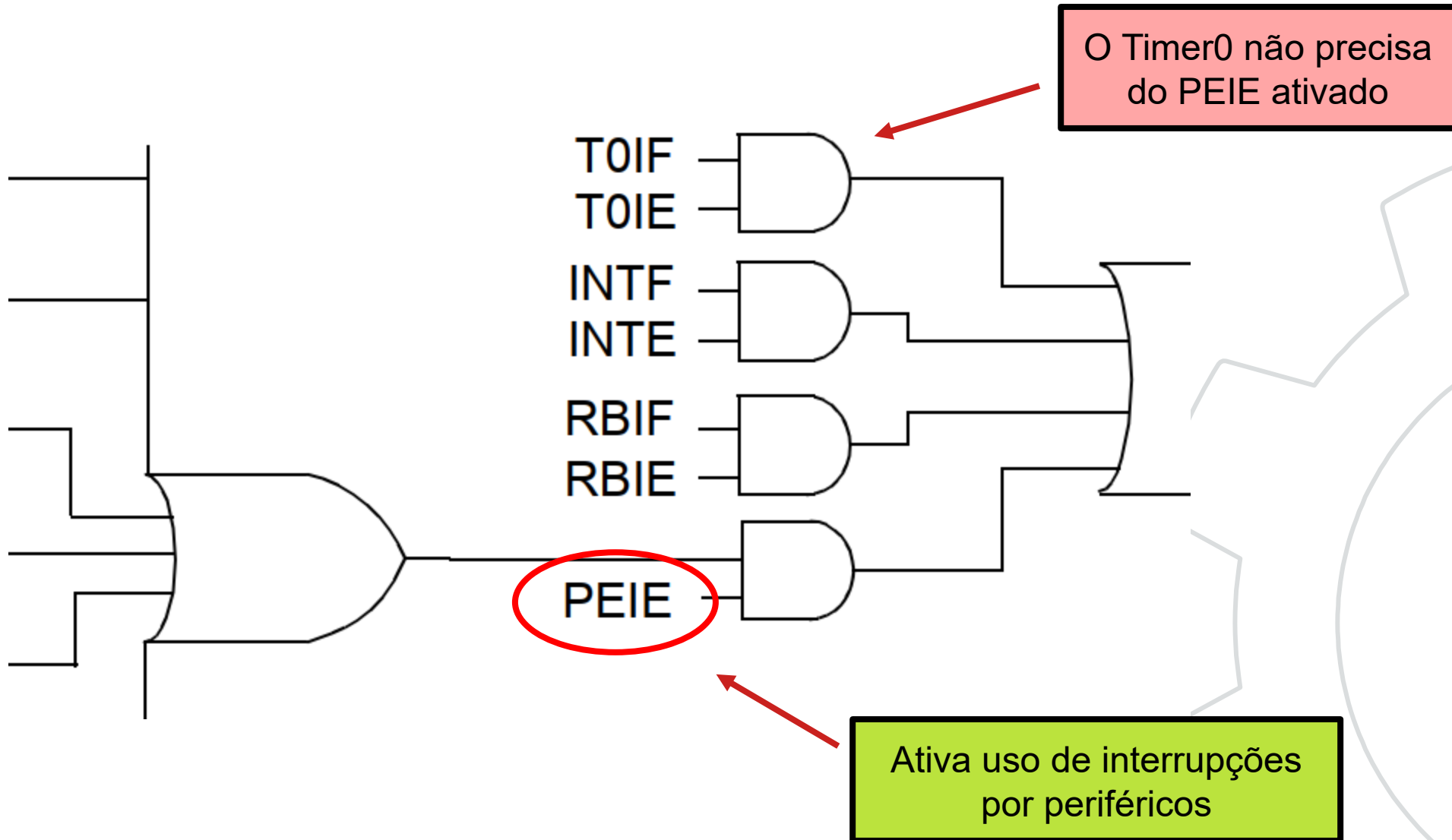
# Exemplo PIC16F628A



# Exemplo PIC16F628A

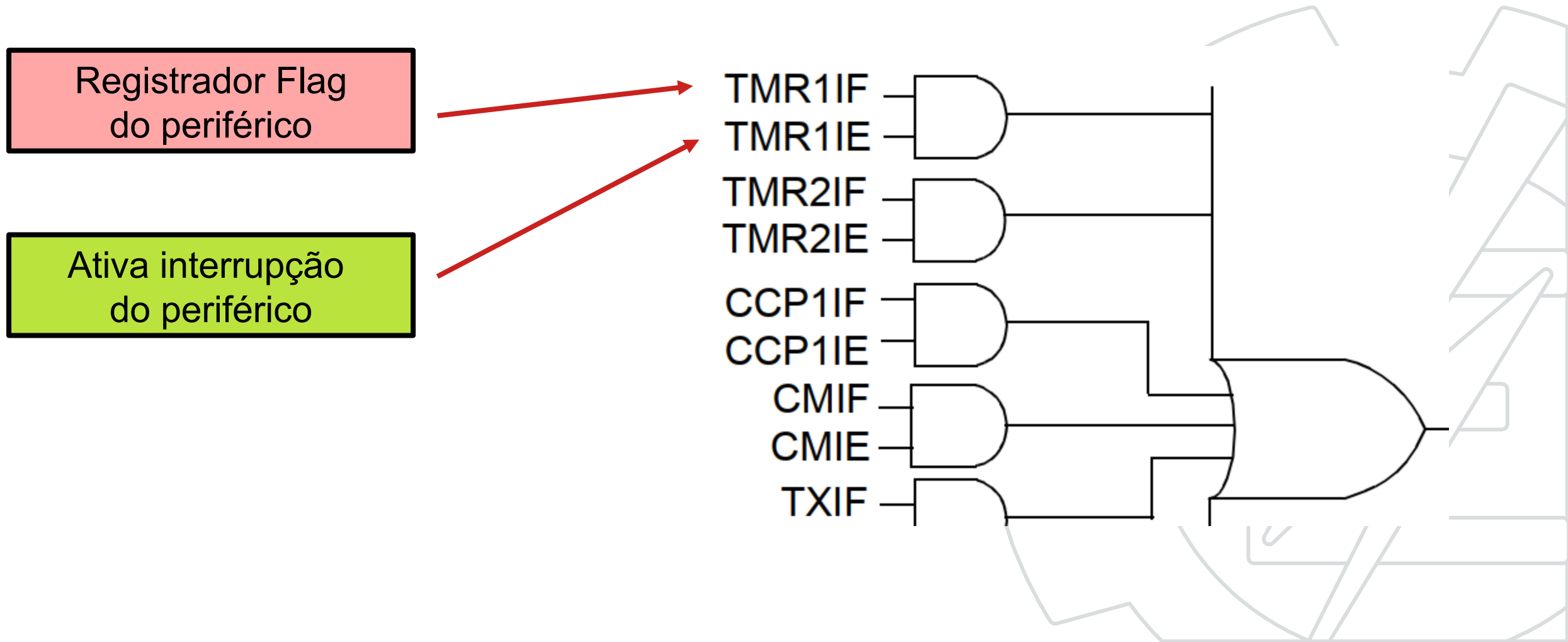


# Exemplo PIC16F628A



# Exemplo PIC16F628A

O Registrador FLAG é ativado independente se vamos usar interrupção ou não.



# Exemplo PIC16F628A

Nas aulas anteriores utilizamos o registrador Flag sem usar a interrupção

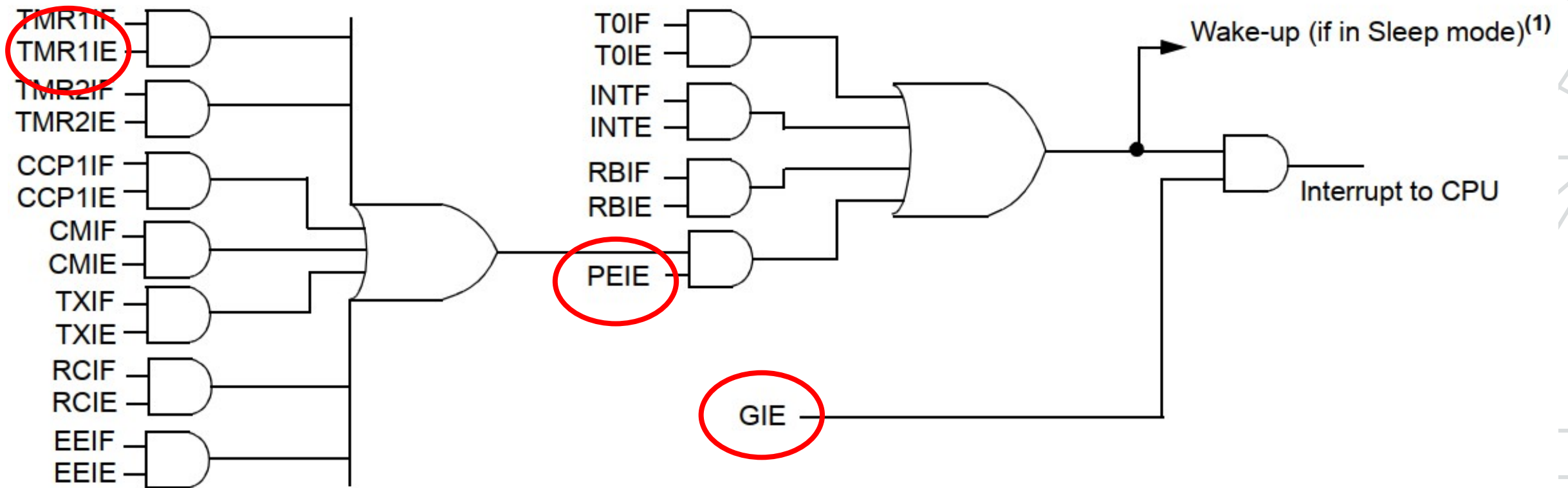
```
...  
int cont=31; //ajustando para 31 estouros  
ajusta_timer();  
For(;;){  
    if(T0IF){ //estourou  
        T0IF = 0; //reseta o contador  
        cont--; //decrementa o contador de estouros  
        if(cont==0){  
            ...  
            cont=31;  
        }  
    }  
    ... //executa outros comandos  
}
```

Flag do TIMER0



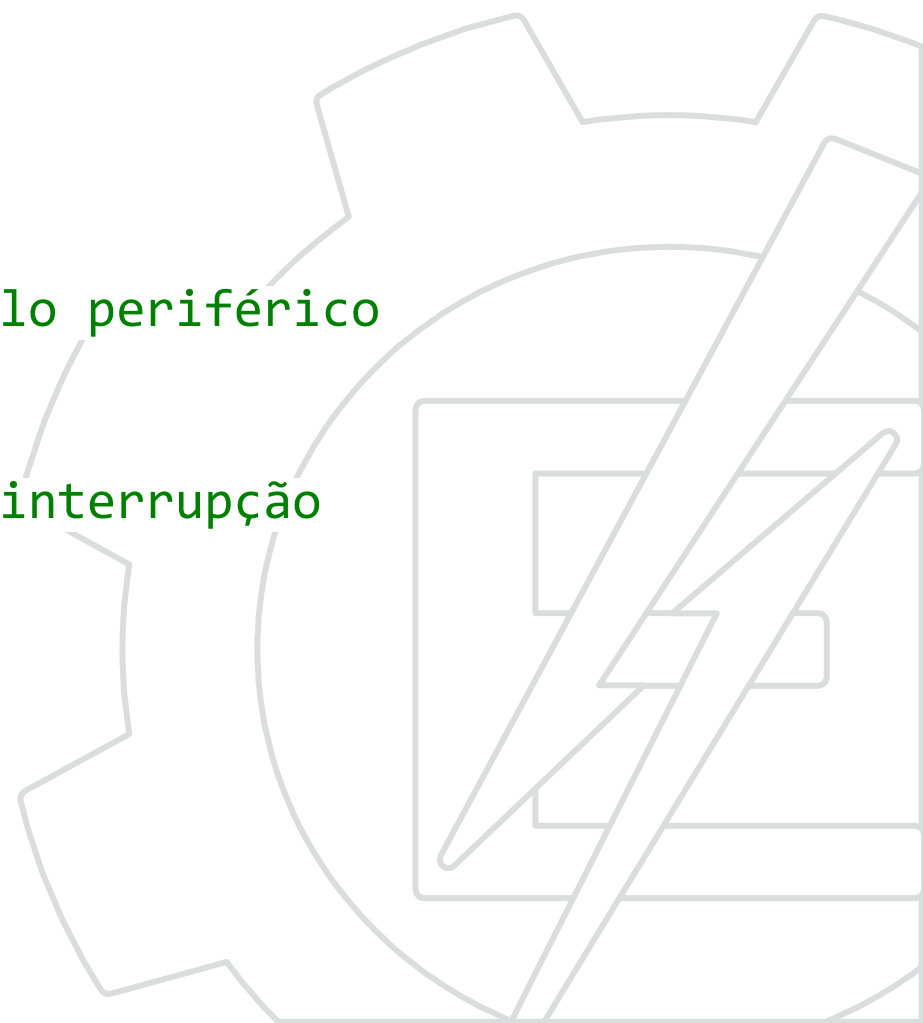
# Exemplo PIC16F628A

Resumindo: Para utilizar a interrupção do periférico TIMER1 temos que ajustar três registradores.



# Exemplo: Porta Serial no PIC18F4520

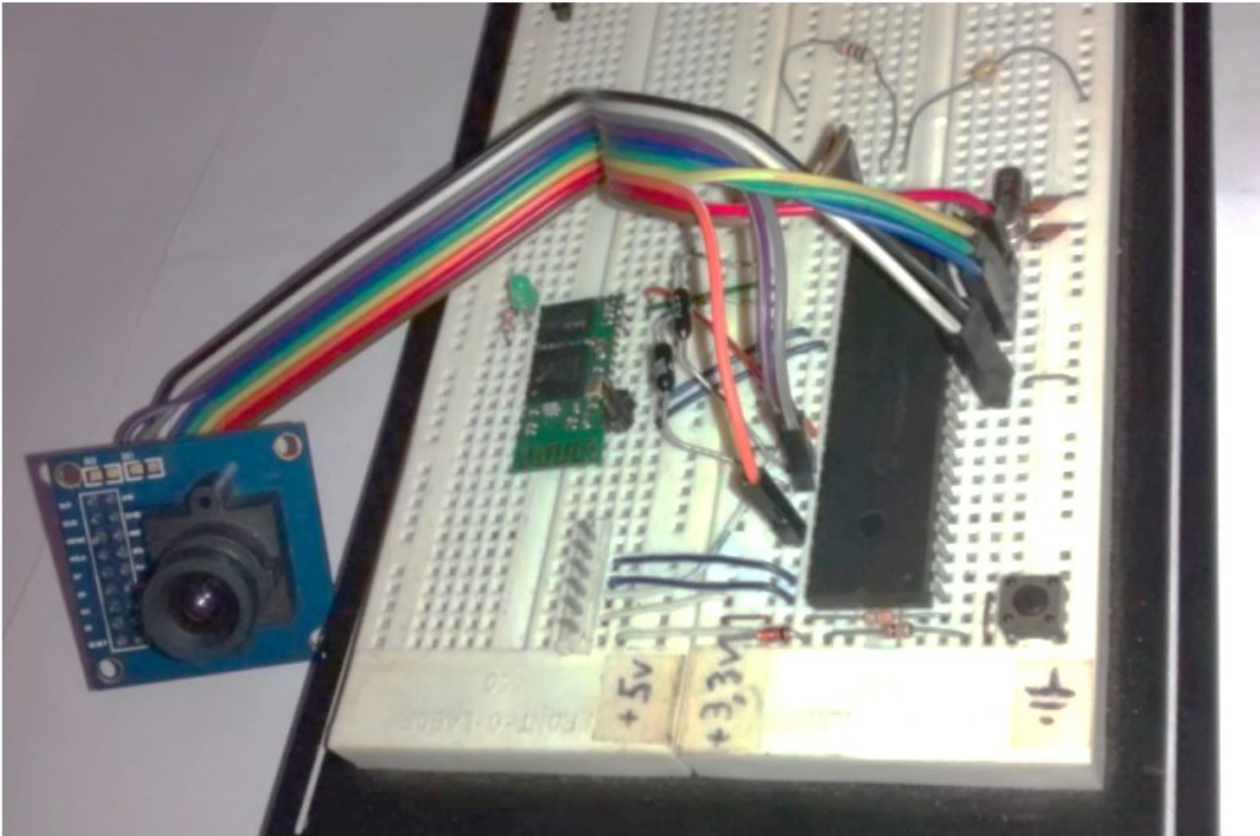
```
void main(void) {  
    systemInit();  
  
    // Inicializa o periférico da serial  
    serialInit();  
  
    // Habilita a geração de uma interrupção pelo periférico  
    UART0_C2 |= 1 << 5;  
  
    // Habilita o microcontrolador a atender a interrupção  
    // gerada pela serial  
    NVIC_ICPR |= 1 << (12 % 32);  
    NVIC_ISER |= 1 << (12 % 32);  
    for (;;) ;  
}
```



# Considerações importantes

- Uma interrupção não é chamada pelo dispositivo portanto não recebe valor como argumento.
- É possível utilizar uma variável global que o programa principal verifica porém não temos controle de quando a interrupção será chamada e mudará o valor.
- O programa principal não sabe que a interrupção ocorreu, se necessário ele deve verificar de alguma forma. Exemplo: porta serial recebeu um byte.

# Projeto de sistema baseado em interrupções

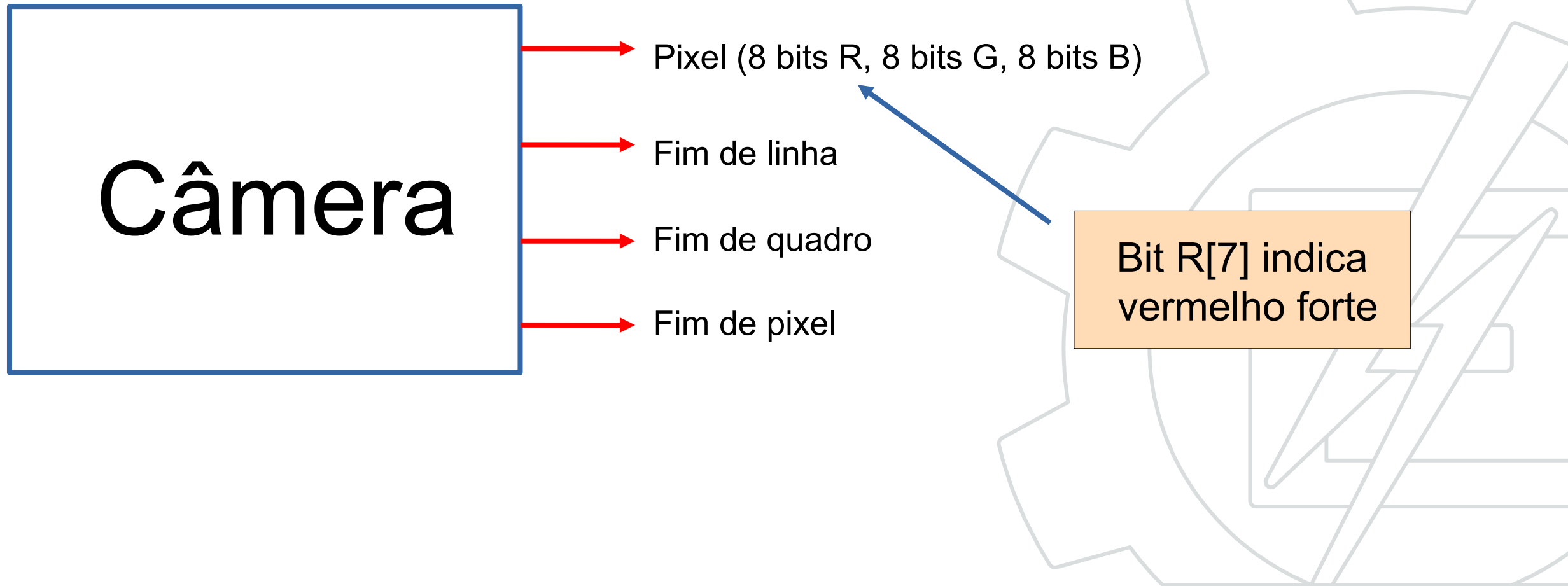


- 30 frames por segundo.
- Imagem RGB 640x480.
- Uma Imagem 921kbytes.
- 27.640.000 bytes por segundo.



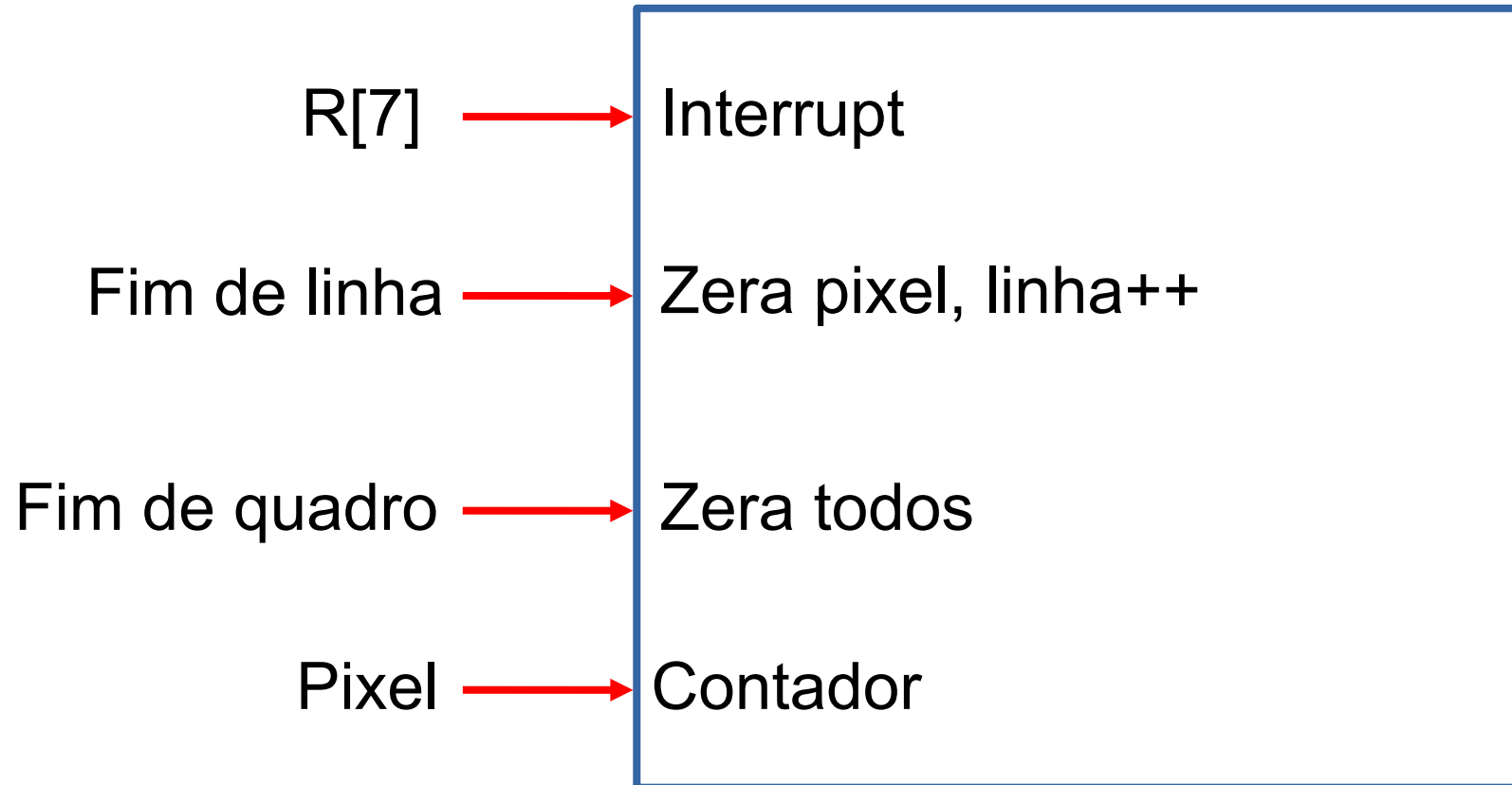
X,Y

# Projeto de sistema baseado em interrupções

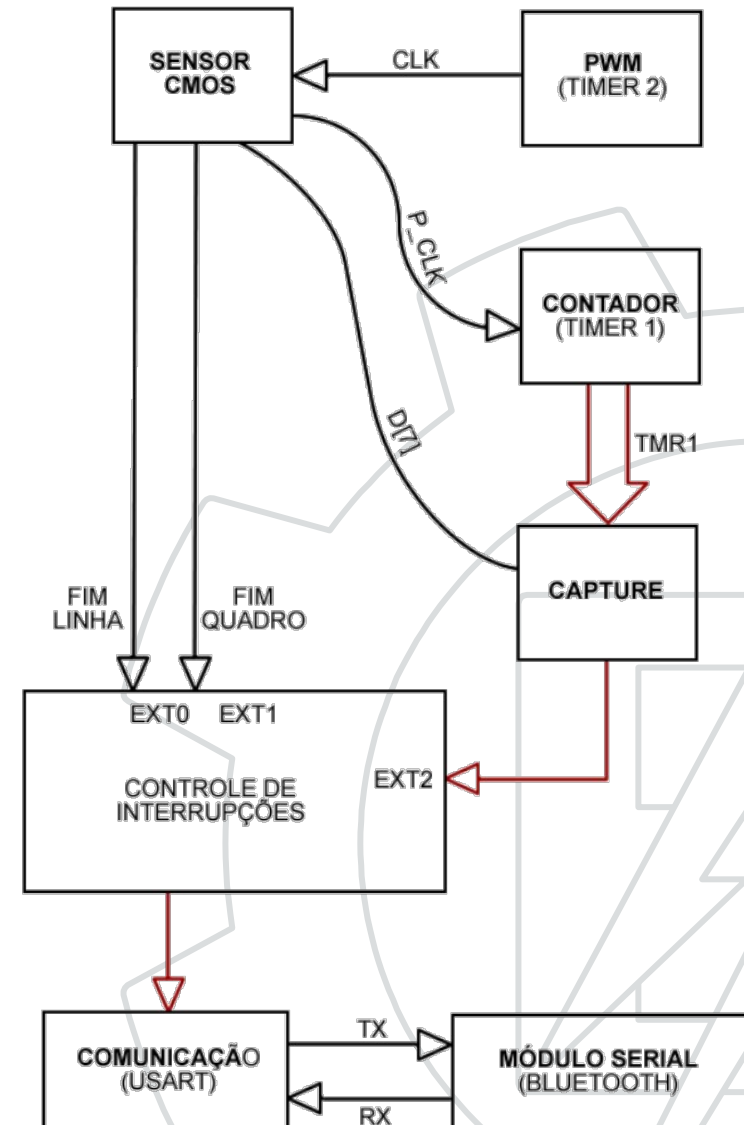
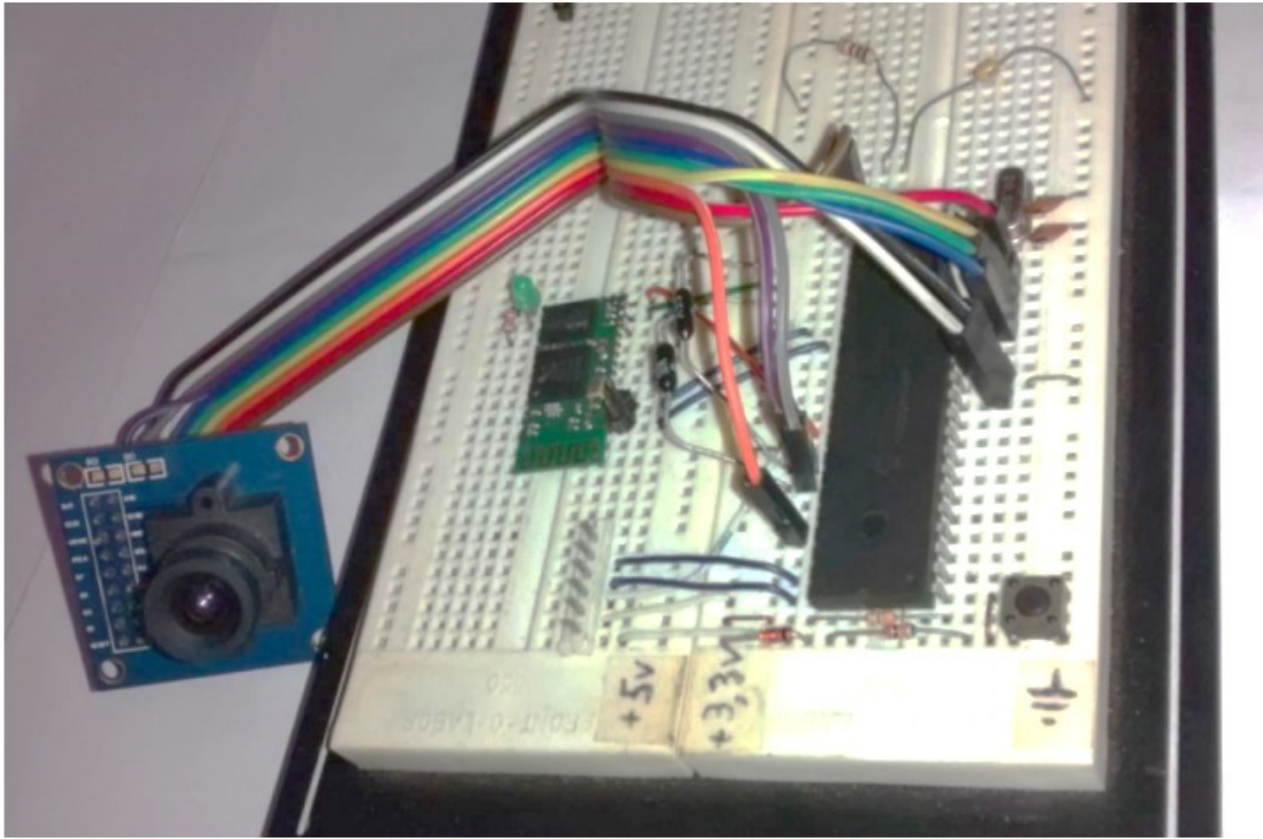


# Projeto de sistema baseado em interrupções

## Microcontrolador



# Projeto de sistema baseado em interrupções



# Projeto de sistema baseado em interrupções

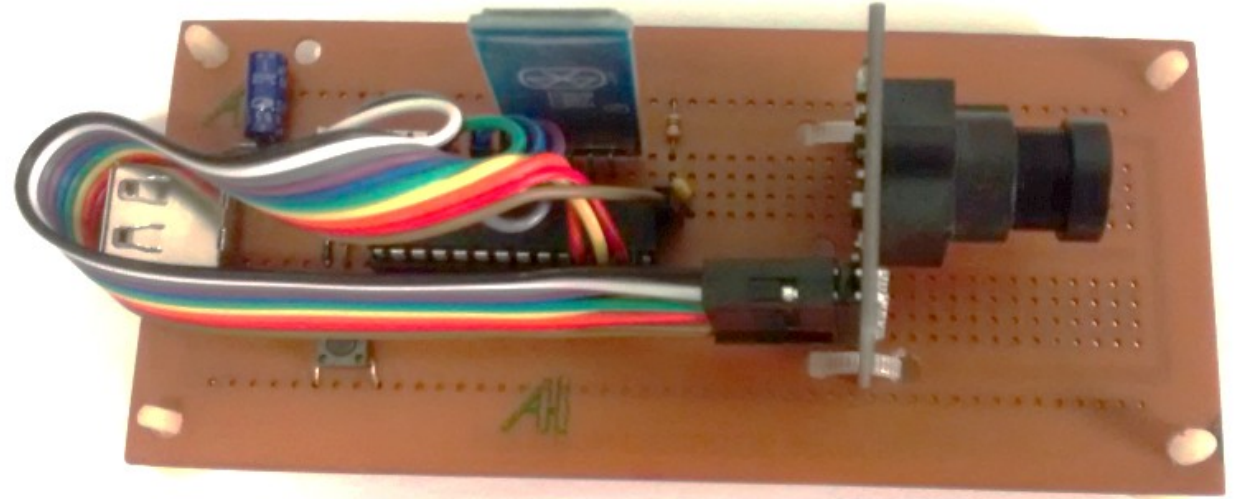
```
void interrupt interrupcao ( void ) {  
    if ( IRQ_Pixel && flagX ){ // Interrupção externa (red pixel > 127)  
        x0=CCPR1 ; //copia coluna do pixel encontrado  
        if ( x0 >0){  
            flagX =0; flagY =0;  
        }  
    }  
    if ( IRQ_Linha ){ // Fim de linha  
        IRQ_Linha = 0 ;  
        if ( flagX==0 && flagY==0){  
            x1=CCPR1 ;  
            if ( x1>x0 ) x=x0+(x1-x0 ) / 2 ;  
            if ( x1==x0 ) x=x0 ;  
            y0=linha ; //Copia a linha que os pixels foram capturados  
            flagY =1;  
        }  
        if ( CCPR1 >0) y1=linha ;  
        linha++; // Incrementa a linha a cada chamada da funcao  
        TMR1 =0;  
        CCPR1 =0;  
    }  
}
```

# Projeto de sistema baseado em interrupções

```
}  
  
if ( IRQ_Frame && PORTBbits.RB4==1){ // IRQ fim de novo frame  
    IRQ_Frame =0; IRQ_Pixel =0; IRQ_Linha =0;  
    if ( y1>y0 ) y=y0+(y1-y0 ) / 2 ; // filtro dos pontos  
    if ( y1==y0 ) y=y0 ;  
    if ( yAnterior >0) y=(yAnterior+y ) / 2 ;  
    if ( xAnterior >0) x=(xAnterior+x ) / 2 ;  
    if ( ( xAnterior^x ) < toleranciaX ){ x=xAnterior ;}  
    if ( ( yAnterior^y ) < toleranciaY ){ y=yAnterior ;}  
    if ( x0 >0){ // se encontrou algo, envia via serial  
        USARTWriteByte ( ' # ' ) ;  
        // Envia y  
        itoa ( buffer , y , 10 ) ;  
        USARTWriteString ( buffer ) ;  
        USARTWriteString ( "," ) ;  
        // Envia x  
        itoa ( buffer , x , 10 ) ;  
        USARTWriteString ( buffer ) ;  
        USARTWriteByte ( '\\n' ) ;  
    }  
    yAnterior=y ; xAnterior=x ;  
    linha =0; x=0; x0 =0; x1 =0;  
    y=0; y0 =0; y1 =0;  
    CCPR1 =0; TMR1 =0;  
    flagX =1; flagY =1;  
}
```

# Projeto de sistema baseado em interrupções

```
33 void main ( ) {  
34     // Direcao e estado dos pinos do PIC  
35     TRISCbits.RC0 = Entrada ;  
36     TRISBbits.RB4 = Entrada ;  
37     TRISAbits.RA0 = Saida ;  
38     PORTAbits.RA0 = Ligado ;  
39  
40     USARTInit ( ) ;  
41     OpenPWM2 ( 0 ) ;  
42     SetDCPWM2 ( 2 ) ;  
43  
44     InicializarI2C ( ) ;  
45     ConfigurarSensor ( ) ;  
46     // Configuracao do Timer 1 -----  
47     T1CON = 0b1001010;  
48     // Configuracao do CCP -----  
49     T3CONbits= 0x02;  
50     CCP1CON = 0b0110;  
51     INTCONbits.RBIE = 1;  
52     INTCON2bits.INTEDG2 = 0;  
53     INTCON3bits.INT2E = 1;  
54     // Interrupções -----  
55     T1CONbits.TMR1ON = 1;  
56     PIE1bits.CCP1IE = 1;  
57     RCONbits.IPEN = 0;  
58     INTCONbits.PEIE = 1;  
59     INTCONbits.GIE = 1;  
60     flagX =1; flagY =1;  
61  
62     while ( 1 ) ; // Processo principal VAZIO !!!  
63 }
```



# Watchdog



# Problemas inesperados com software

- Um programa pode parar de funcionar nos seguintes casos:
  - Loop infinito por erro.
  - Dead lock.
  - Hack.
  - Bugs/erros nos algoritmos.



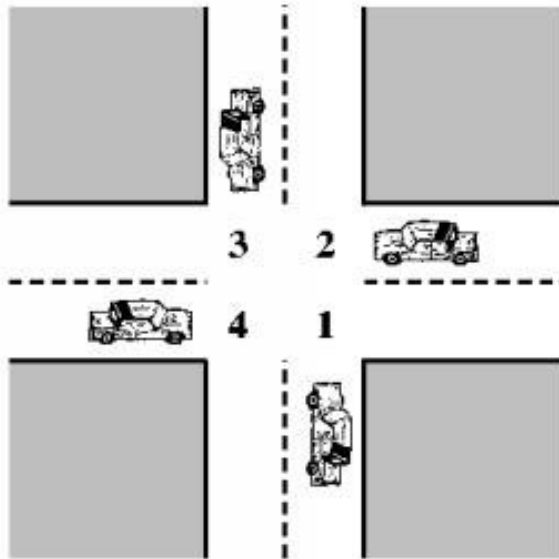
# Loop infinito por erro



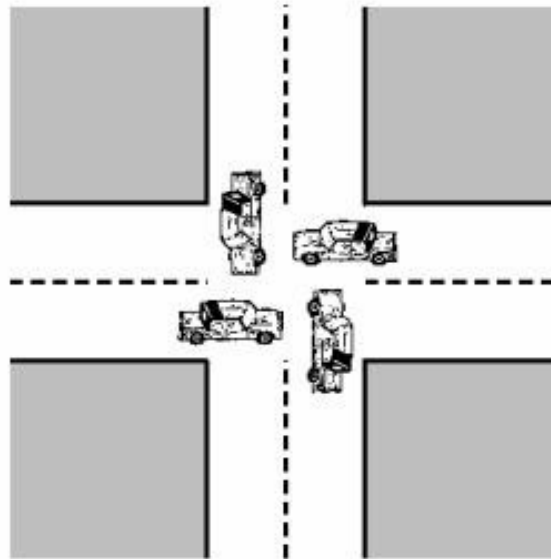
Tem que  
resetar!!!



# Deadlock



(a) Deadlock possible



(b) Deadlock

Duas estruturas podem querer usar um periférico ao mesmo tempo

# Deadlock

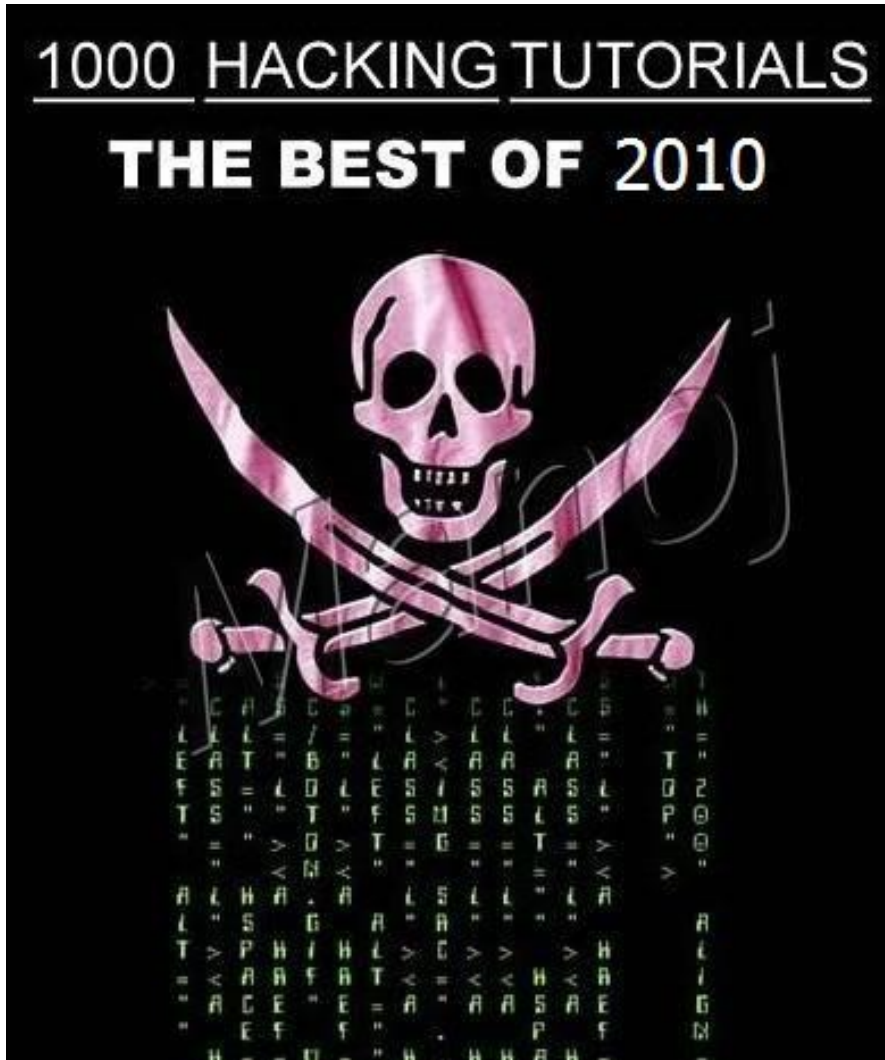


Um cruzamento é  
um local  
compartilhado

Ocorre principalmente em  
sistemas com tempo real

- Brigadeiro Faria Lima x Presidente Juscelino (SP)
  - Foto de Silvio Tanaka

# Hack



Sua cafeteira pode ter  
conexão com a sua rede

# Exemplo de "loop infinito"

- Se o botão não for pressionado o while não termina.
- Se o while não terminar a função kpDebounce() não é executada.
- Se a função kpDebounce() não é executada o sistema não percebe que a tecla mudou.
- Se o sistema não perceber que a tecla mudou, o while não termina.

```
void main(void){  
    kpInit();  
    lcdInit();  
    for(;;){  
        kpDebounce();  
        while (kpRead() == 0);  
        lcdPrint(kpRead());  
    }  
}
```

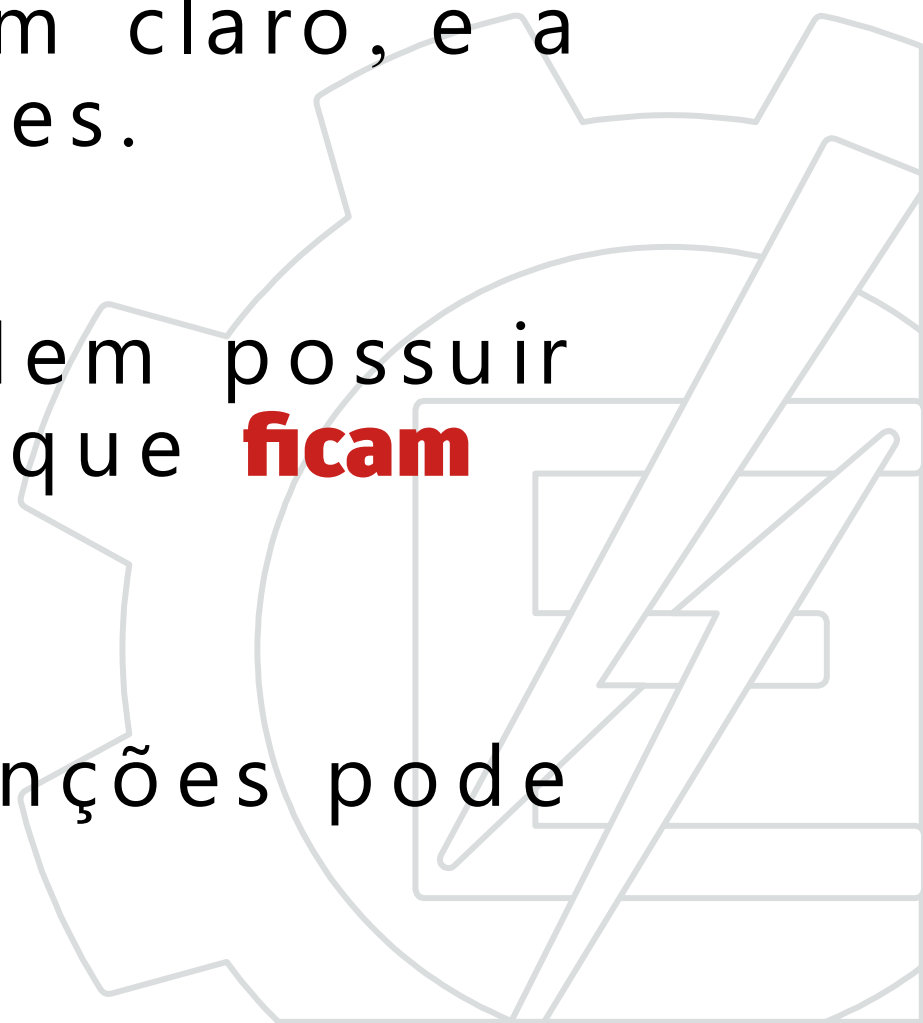
# Solução

- Ao invés de ficar aguardando o pressionamento no while, podemos só perguntar se foi pressionada a cada loop.

```
void main(void){  
    kpInit();  
    lcdInit();  
    for(;;){  
        kpDebounce();  
        if(kpRead() != 0){  
            lcdPrint(kpRead());  
        }  
    }  
}
```

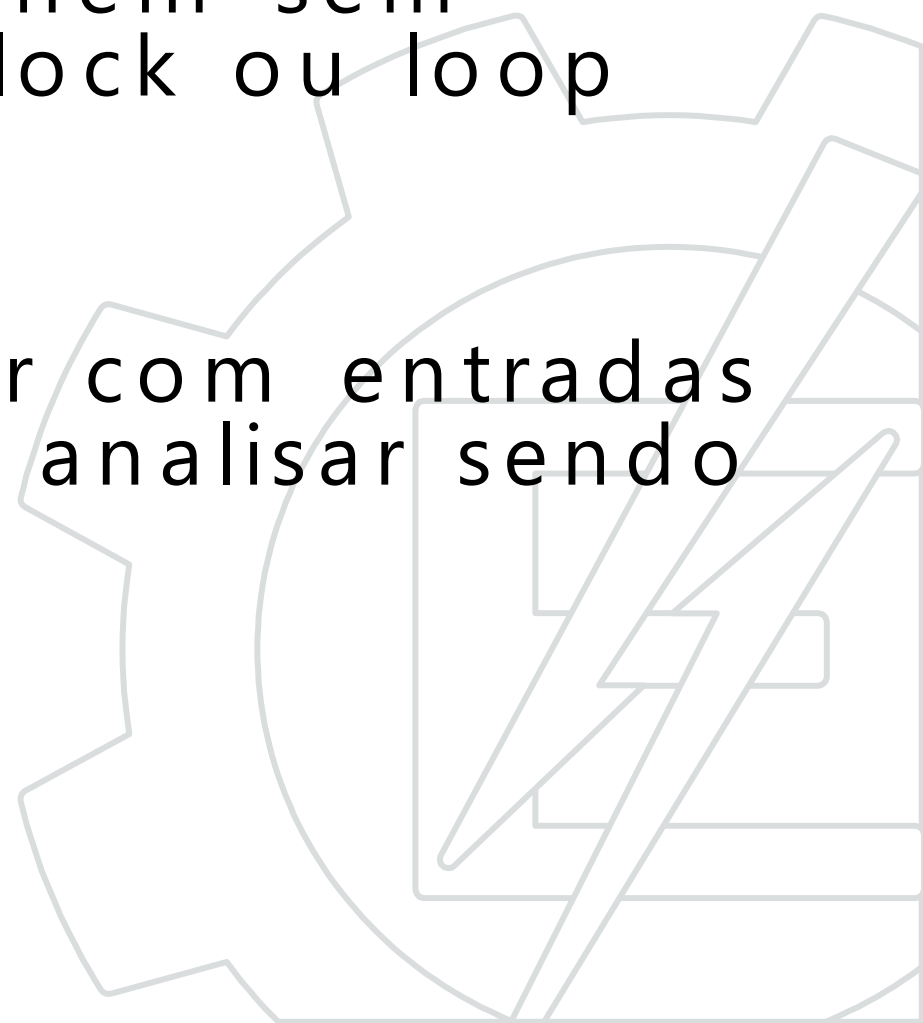


# Os loops infinitos "escondidos"

- O exemplo apresentado é bem claro, e a solução é relativamente simples.
  - Sistemas mais complexos podem possuir interdependência de funções que **ficam escondidas** em sub-rotinas.
  - Até mesmo um loop entre funções pode ocorrer.
- 

# Os loops infinitos "escondidos"

- Garantir que sistemas funcionem sem nenhuma condição de deadlock ou loop infinito pode ser difícil.
- Um sistema que precisa lidar com entradas de usuários é mais difícil de analisar sendo ainda mais complexo.
- **Solução:** “Watchdog”.



# Watchdog

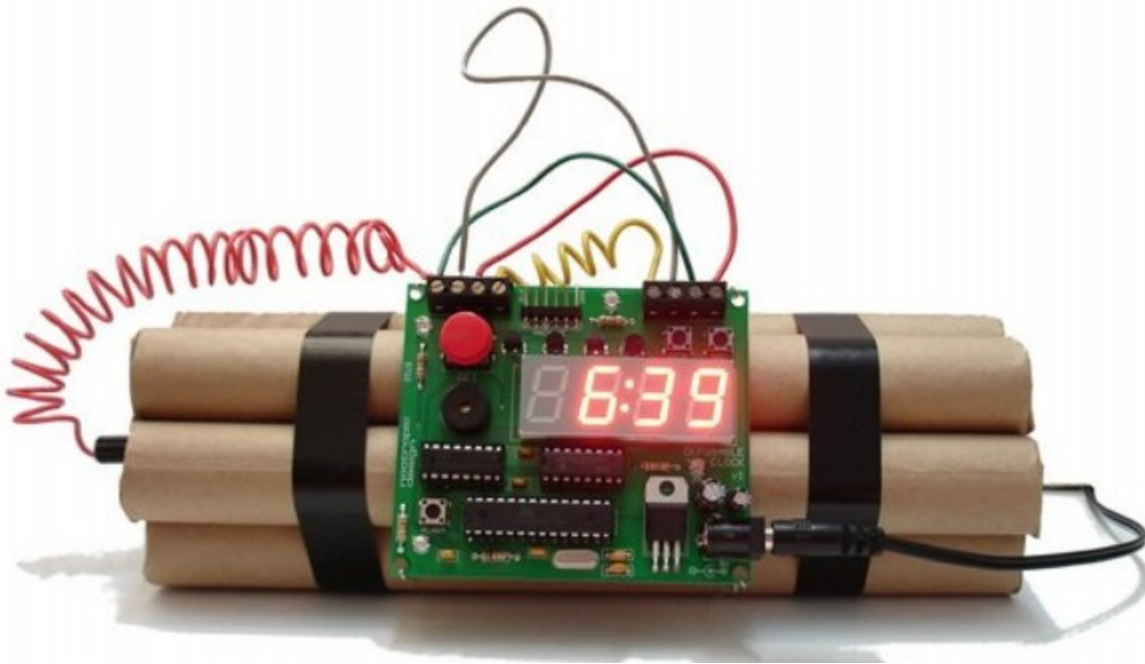
O Watchdog não resolve o problema, ele reseta o microcontrolador para sair do “travamento”.



Visa aumentar a segurança do projeto.

# Watchdog

Funciona como um temporizador que precisa ter seu contador constantemente reiniciado.



Caso não seja reiniciado no tempo exigido, o watchdog reinicia o microcontrolador.

# Watchdog

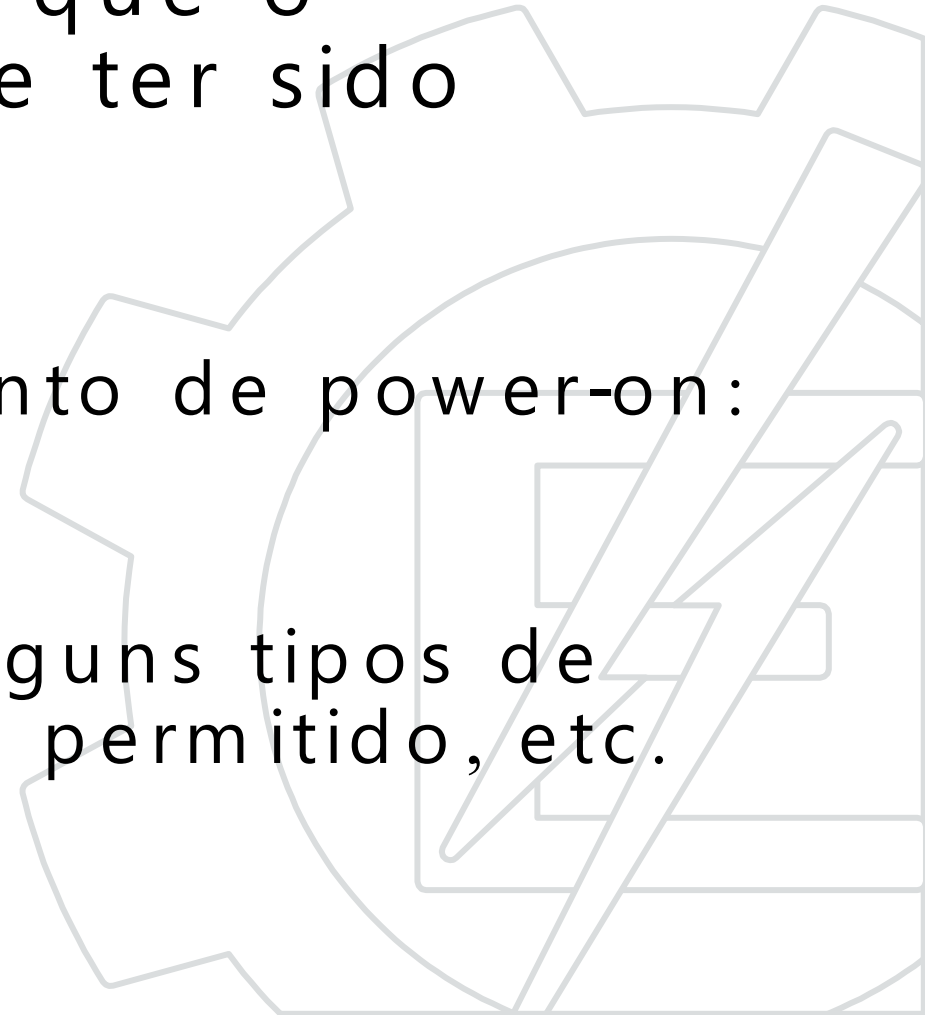
```
void main(void){  
    kpInit();  
    lcdInit();  
    watchdogInit();  
    for(;;){  
        kpDebounce();  
        if (kpRead() != 0){  
            lcdNumber(kpRead());  
        }  
        watchdogFeed();  
    }  
}
```



Reinicia o contador

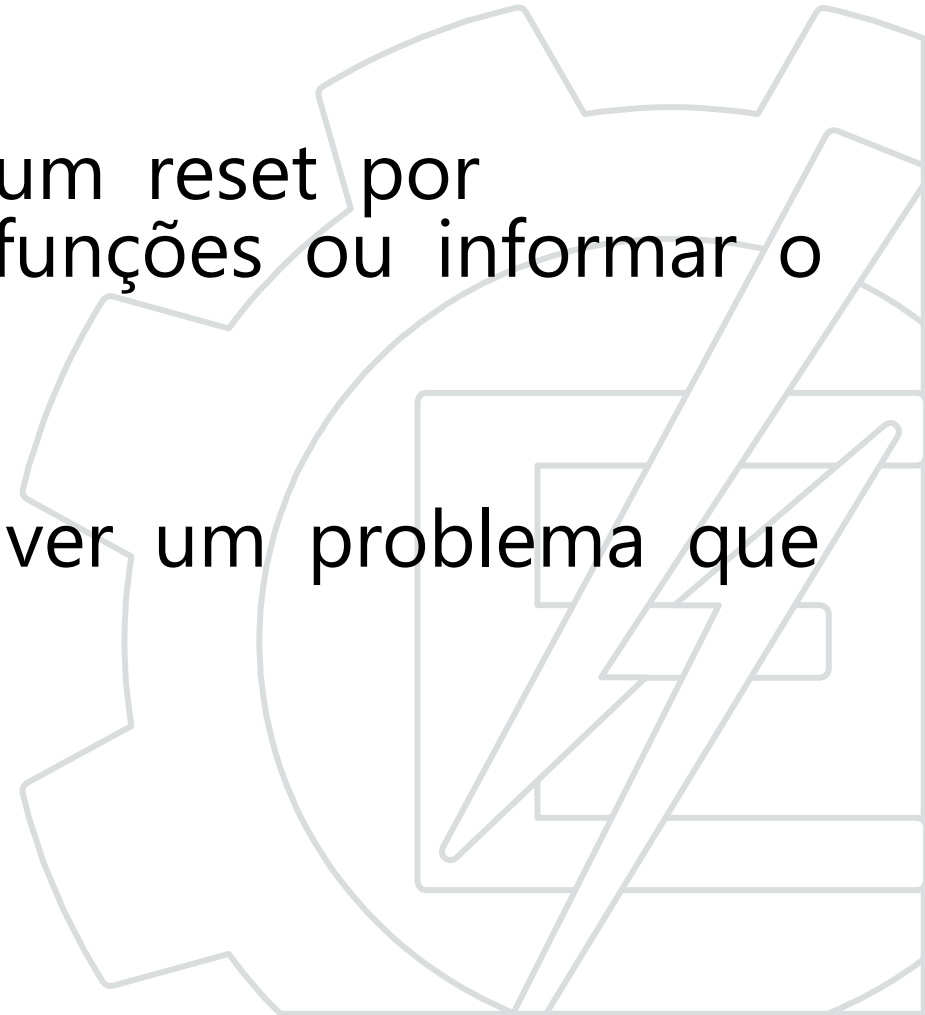
# Watchdog

- Quando o sistema é reiniciado, alguns microcontroladores permitem que o programa saiba o porquê dele ter sido reiniciado.
  - A causa mais comum é um evento de power-on: a placa acabou de ser ligada.
  - Outras causas são: watchdog, alguns tipos de falha, nível de tensão abaixo do permitido, etc.



# Watchdog

- Existe uma grande chance do sistema voltar a funcionar.
- Se o programa for capaz de identificar um reset por watchdog ele pode desabilitar algumas funções ou informar o problema para o operador.
- O Watchdog é a última chance de resolver um problema que não foi previsto.



# Exemplo de verificação do Watchdog

```
void main(void){
    systemInit();
    //primeiro teste antes da continuidade do programa
    if(watchdogProblem()){
        lcdString("Problema 01");
        for(;;); //não pode continuar execução
    }
    pwmInit();
    serialInit();
    lcdInit();
    watchdogInit();

    for(;;){
        serialProcess();
        pwmProcess();           //executar a cada 1ms
        watchdogFeed();
    }
}
```



Até a próxima!

